

Programación 2

TUDAI

Resolución Parcial 2025

Resolución Incremental

Vamos a hacer una solución incremental a medida que la vamos leyendo. La solución sería mas rápida si leemos todo el enunciado y realizamos inicialmente las abstracciones

Parcial Martes 28 de Octubre de 2025

— — —

La municipalidad de Metrópolis te contrata para desarrollar un sistema de gestión de transporte público. El sistema debe registrar distintos tipos de vehículos y ser capaz de calcular el costo del mantenimiento diario de toda la flota de vehículos. Los colectivos poseen una patente, un modelo (año), los kilómetros recorridos y una determinada cantidad de asientos. Para los colectivos urbanos, el costo diario de mantenimiento es de \$5.000 por asiento. Los colectivos de media distancia tienen un costo mayor, de \$7.000 por asiento.

Primer aproximación - Vehículo

Clase vehículo con los 4 atributos mencionados gets, sets y constructor

C Vehiculo
□ String patente □ int modelo □ int asientos □ double kilometros
● Vehiculo(String patente, int modelo, int asientos, double kilometros) ● String getPatente() ● void setPatente(String patente) ● int getModelo() ● void setModelo(int modelo) ● int getAsientos() ● void setAsientos(int asientos) ● double getKilometros() ● void setKilometros(double kilometros)

Solución
Literal hasta
el momento



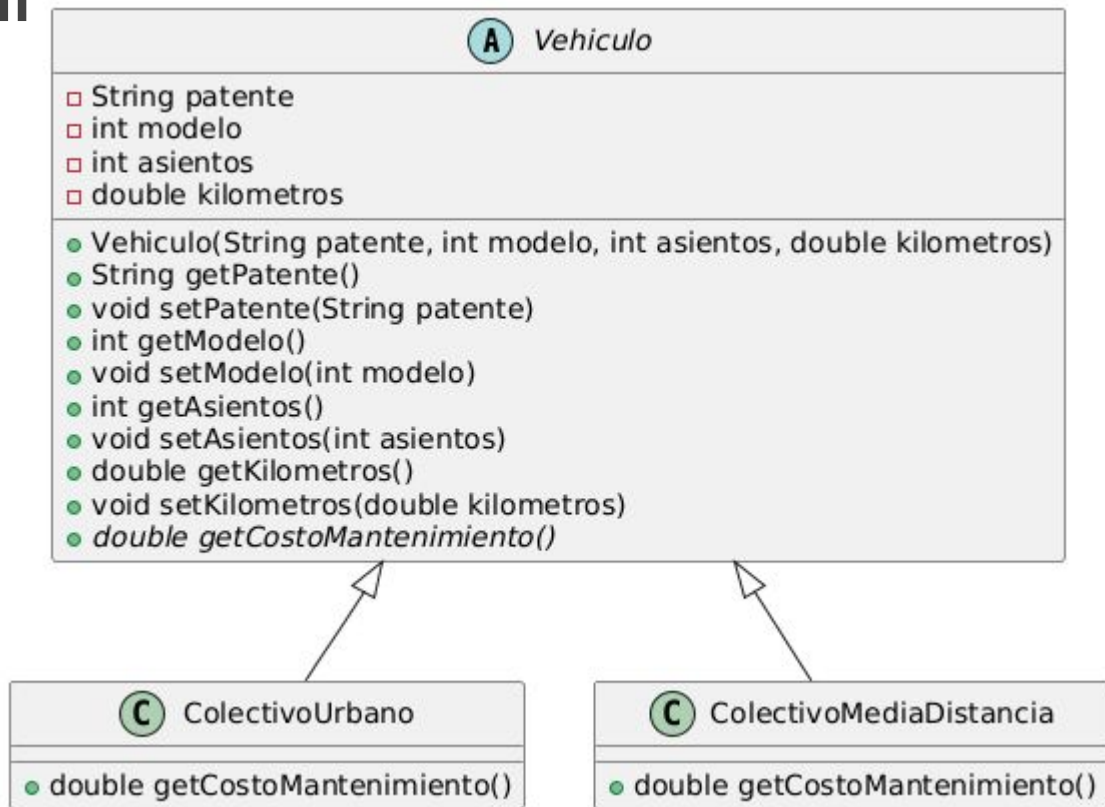
Parcial Martes 28 de Octubre de 2025

— — —

La municipalidad de Metrópolis te contrata para desarrollar un sistema de gestión de transporte público. El sistema debe registrar distintos tipos de vehículos y ser capaz de calcular el costo del mantenimiento diario de toda la flota de vehículos. Los colectivos poseen una patente, un modelo (año), los kilómetros recorridos y una determinada cantidad de asientos. Para los colectivos urbanos, el costo diario de mantenimiento es de \$5.000 por asiento. Los colectivos de media distancia tienen un costo mayor, de \$7.000 por asiento.

Aumentamos la solución

Costo de Mantenimiento
no lo se resolver en
Vehículo lo dejo
abstracto, la clase es
abstracta y heredan los
dos colectivos



Aumentamos la solución

Costo de Mantenimiento
no lo se resolver en
Vehículo lo dejo
abstracto, la clase es
abstracta y heredan los
dos colectivos

```
public double getCostoMantenimiento() {  
    return 5000 * this.getAsientos();  
}
```

```
public double getCostoMantenimiento() {  
    return 7000 * this.getAsientos();  
}
```

- String patente
- int modelo
- int asientos
- double kilometros

- Vehiculo(String patente, int modelo, int asientos, double kilometros)
- String getPatente()
- void setPatente(String patente)
- int getModelo()
- void setModelo(int modelo)
- int getAsientos()
- void setAsientos(int asientos)
- double getKilometros()
- void setKilometros(double kilometros)

Mantenimiento()



ColectivoUrbano

- double getCostoMantenimiento()



ColectivoMediaDistancia

- double getCostoMantenimiento()



Aumentamos la solución

Costo de Mantenimiento
no lo se resolver en
Vehículo o de
abstract la clase es
abstract y heredan los
dos colectivos

```
public double getCostoMantenimiento() {  
    return 50 * this.getAsientos();  
}
```

```
public double getCostoMantenimiento() {  
    return 7000 * this.getAsientos();  
}
```

- String patente
- int modelo
- int asientos
- double kilometros

Vehiculo(String patente, int modelo, int asientos, double kilometros)

String getPatente()

String setPatente(String patente)

int getModelo()

void setModelo(int modelo)

int getAsientos()

void setAsientos(int asientos)

double getKilometros()

void setKilometros(double kilometros)

ColectivoUrbano

ColectivoMediaDistancia

double getCostoMantenimiento()

double getCostoMantenimiento()



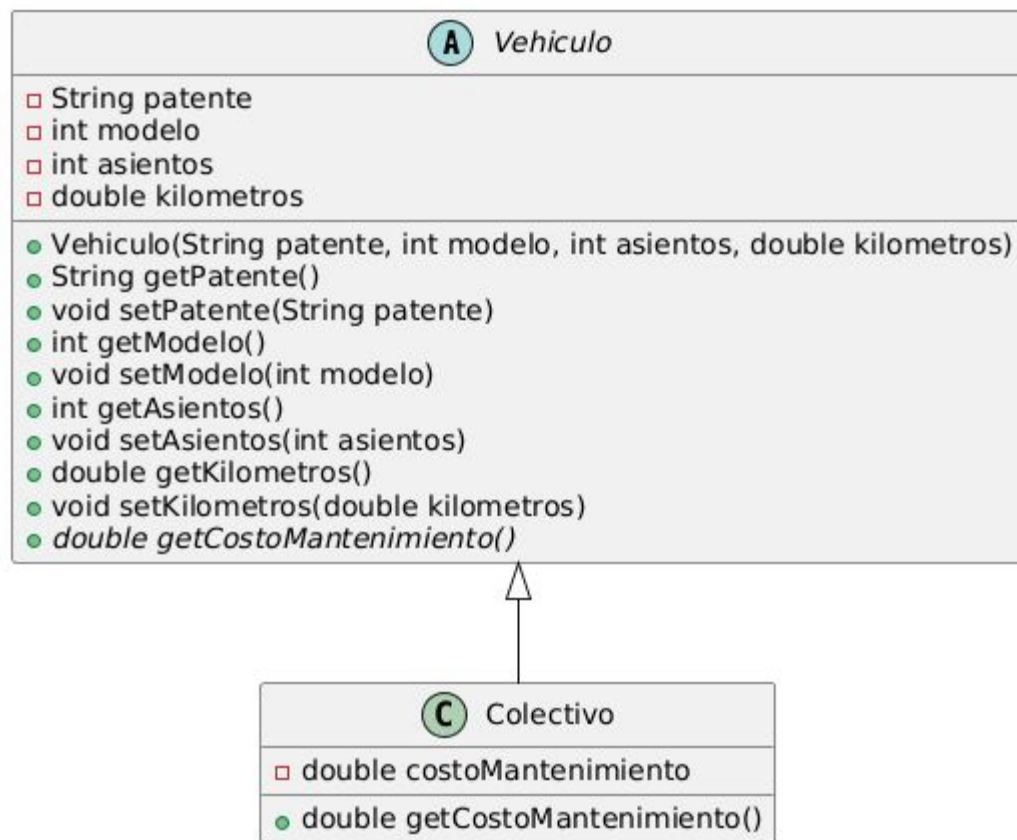
Problemas de la Solución Anterior

- **Constantes en Código.** El 5000 y 7000 son constantes, es decir que si pongo 9000 tengo que hacer una clase nueva. Un valor nuevo es un nuevo desarrollo?
- **Clase Vs Instancia,** si miramos ambas clases son iguales ambas devuelven un valor (una 5000 y otra 7000) por la cantidad de asientos

Es necesario usar variables para el valor (ya vimos que para uno es 5000 y otros 7000)

Corregimos

— — —



Corregimos

— — —

A Vehiculo
□ String patente □ int modelo □ int asientos □ double kilometros
● Vehiculo(String patente, int modelo, int asientos, double kilometros) ● String getPatente() ● void setPatente(String patente) ● int getModelo() ● void setModelo(int modelo) ● int getAsientos() ● void setAsientos(int asientos) ● double getKilometros() ● void setKilometros(double kilometros)

```
public double getCostoMantenimiento() {  
    return costoMantenimiento * this.getAsientos();  
}
```

B Vehiculo
□ double costoMantenimiento ● double getCostoMantenimiento()

Continuamos

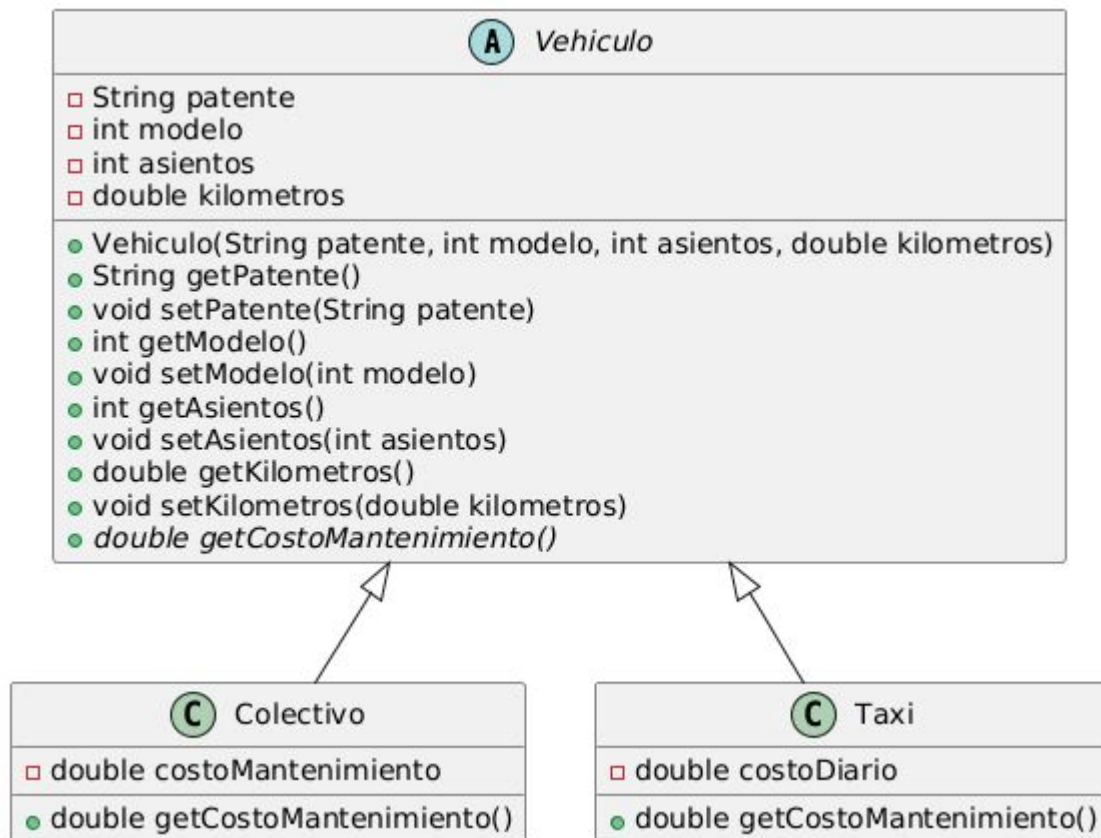
De los taxis se conoce su patente, año del vehículo (modelo), cantidad de asientos disponibles y la cantidad de kilómetros recorridos. El costo de mantenimiento diario es de \$2.000 por cada kilómetro recorrido. A este costo se le suma un extra de \$500, si el año del vehículo es anterior a 2010.

Ya tengo Vehiculo que tiene todos los mismos datos y en este caso taxi solo cambia la forma de calcular el costo (que la dejamos abstracta en vehiculo)



Aumentamos

— — —



Aumentamos

A Vehiculo

```
public double getCostoMantenimiento() {  
    double valor = costoDiario * this.getKilometros();  
    if (this.getModelo() < 2010)  
        valor = valor + 500;  
    return valor;  
}
```



• void setKilometros(double kilometros)
• double getCostoMantenimiento()

C Colectivo

□ double costoMantenimiento
• double getCostoMantenimiento()



C Taxi

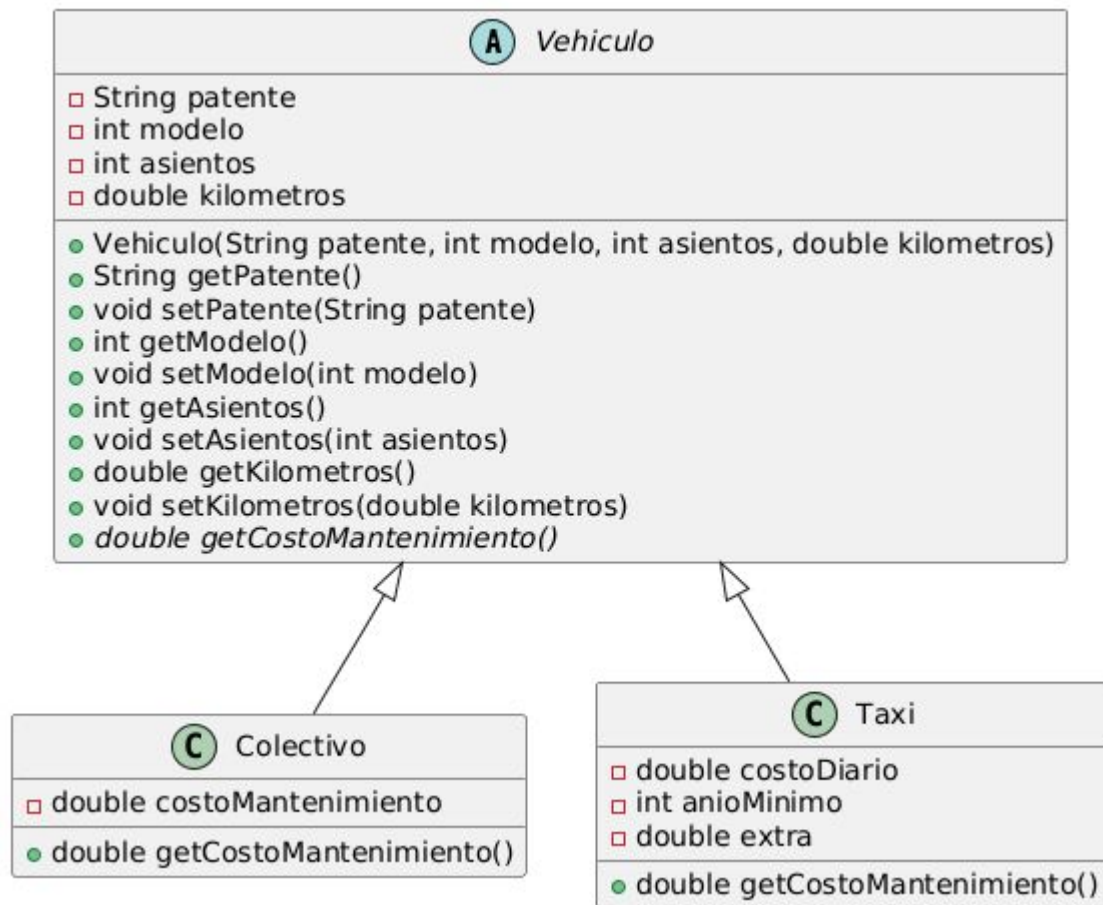
□ double costoDiario
• double getCostoMantenimiento()

Errores

- De nuevo constantes en código, no se generaron sub clases pero cuando avancemos y leamos los remises si no corregimos el error va a quedar una clase nueva por 2024

Corregimos

— — —



Corregimos

A Vehiculo
□ String patente □ int modelo □ int asientos □ double kilometros

```
public double getCostoMantenimiento() {  
    double valor = costoDiario * this.getKilometros();  
    if (this.getModelo() < anioMinimo)  
        valor = valor + extra;  
    return valor;  
}
```



C Colectivo
□ double costoMantenimiento ● double getCostoMantenimiento()

C Taxi
□ double costoDiario □ int anioMinimo □ double extra ● double getCostoMantenimiento()

Avanzamos

Metrópolis decidió organizar su transporte público en flotas. Una flota es un conjunto de vehículos agrupados (puede incluso que tenga flotas menores dentro, así como vehículos). Si a una flota se le consulta el modelo, responde con el modelo más antiguo de los elementos que contiene. La cantidad de asientos de la flota queda determinada por la suma de asientos de sus elementos. Los kilómetros de una flota son la suma de kilómetros de sus elementos. Similarmente, el costo de mantenimiento es la suma de los costos de mantenimiento.

Preguntas Filosóficas

- La flota tiene cantidad de asientos?

La flota no tiene cantidad de Asientos, pero si se le puede preguntar por su cantidad y la calcula. Tiene un `getAsientos`



Preguntas Filosóficas

- La flota tiene Kilometros?

La flota no tiene kilometros, pero si se le puede preguntar por su cantidad de kilometros y realiza una suma. Tiene un `getKilometros`

Esto va a ser similar para `Modelo` y para `costo de mantenimiento`, no tiene los atributos pero si los gets



Preguntas Filosóficas

- La flota ES UN vehículo?

No es un Vehículo, tiene cosas en común pero no ES un vehículo



Preguntas Filosóficas

- La flota tiene vehiculos?



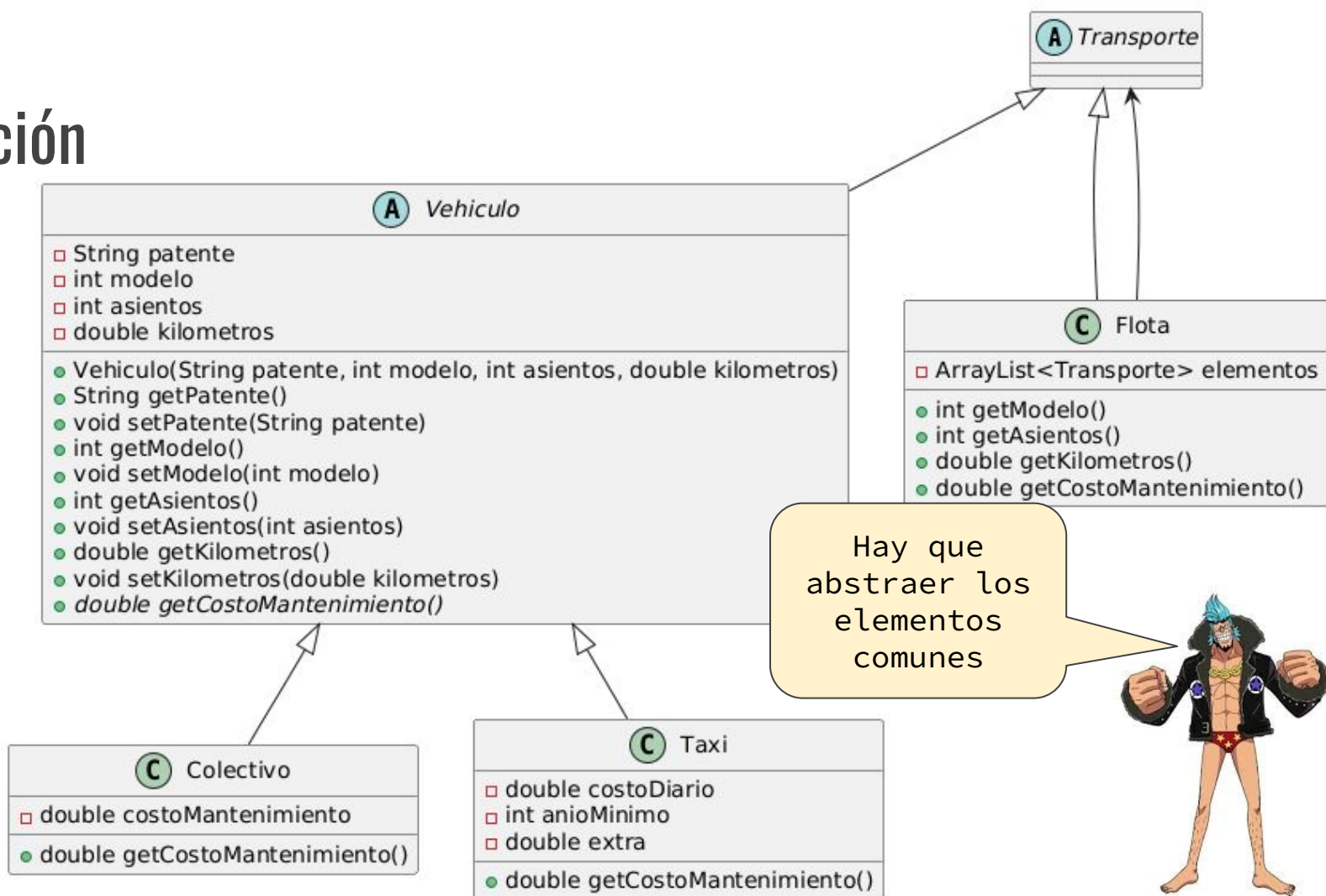
Preguntas Filosóficas

- La flota tiene flotas?

Tiene ambas cosas, creo una clase común entre vehículo y flota en donde subo las cosas comunes.



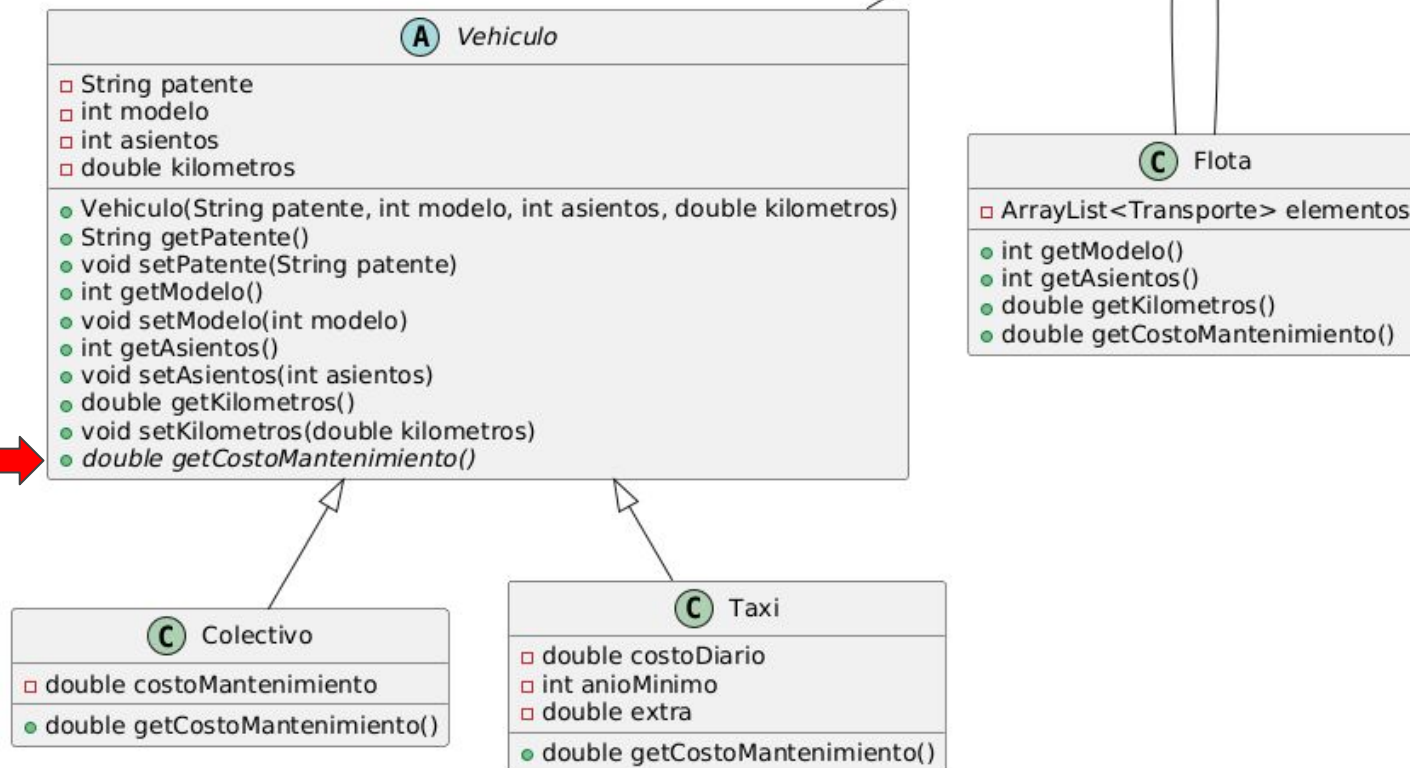
Ampliación



Abstracción

Métodos comunes pero no implementaciones o atributos

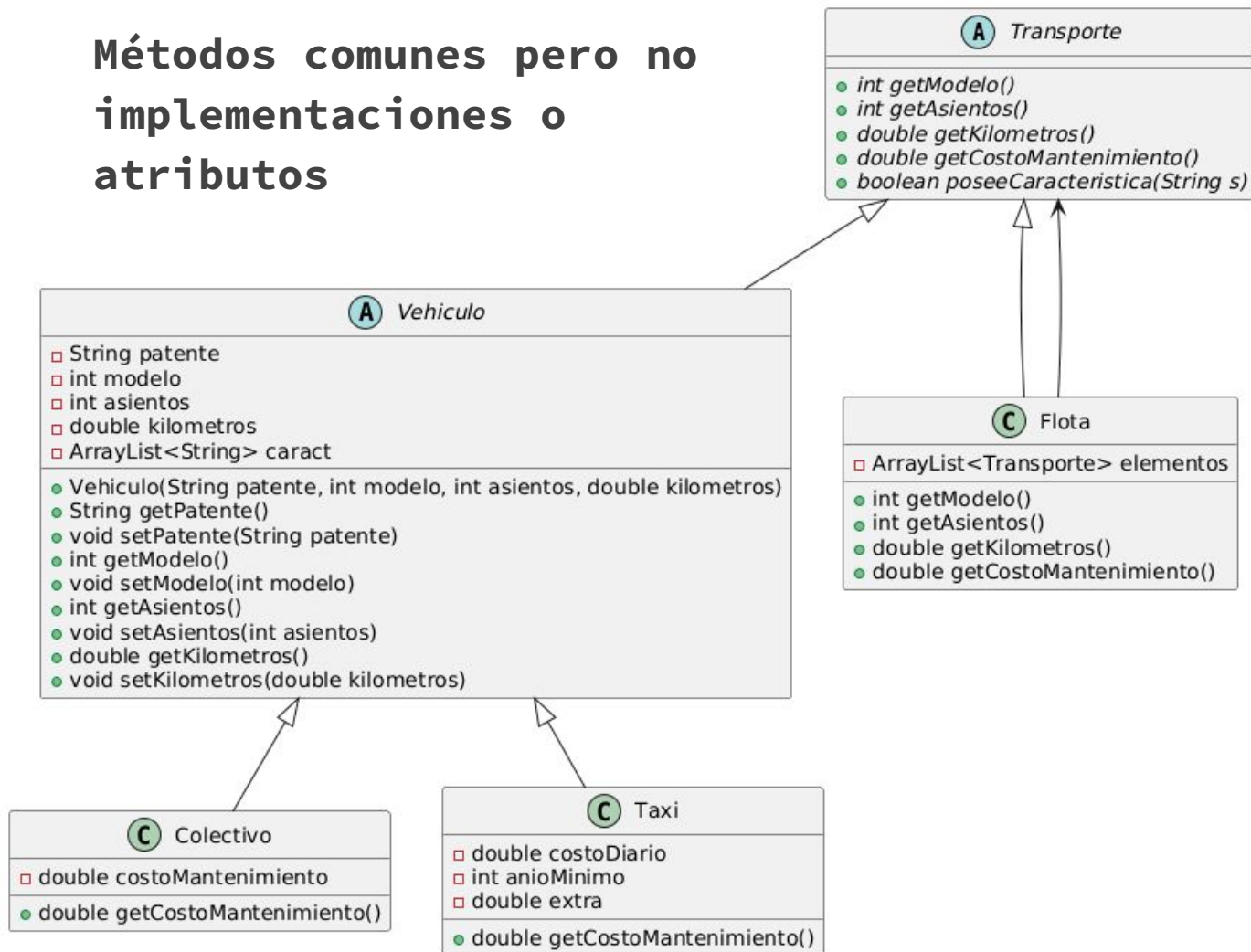
No es necesario repetirlo como abstracto (vehículo es abstracta)



Abstracción

Métodos comunes pero no implementaciones o atributos

No repetimos el abstracto



Flota: getModelo()

```
public int getModelo() {  
    int modeloMenor = Integer.MAX_VALUE;  
    for(Transporte ee : elems) {  
        int actual = ee.getModelo();  
        if (actual < modeloMenor) {  
            modeloMenor = actual;  
        }  
    }  
    return modeloMenor;  
}
```

Flota: getAsientos

```
public int getAsientos() {  
    int suma = 0;  
    for(int i = 0; i < elementos.size(); i++) {  
        suma = suma + elementos.get(i).getAsientos();  
    }  
    return suma;  
}
```

Flota: getAsientos

```
public int getAsientos() {  
    int suma = 0;  
    for(int i = 0; i < elementos.size(); i++) {  
        suma = suma + elementos.get(i).getAsientos();  
    }  
    return suma;  
}
```

Flota: getCostoMantenimiento

```
public double getCostoMantenimiento() {  
    double suma = 0;  
    for(int i = 0; i < elementos.size(); i++) {  
        suma = suma + elementos.get(i).getCostoMantenimiento();  
    }  
    return suma;  
}
```

Flota: getKilometros

```
public double getKilometros() {  
    double suma = 0;  
    for(int i = 0; i < elementos.size(); i++) {  
        suma = suma + elementos.get(i).getKilometros();  
    }  
    return suma;  
}
```

Continuamos

Para facilitar la descripción, cada vehículo debe contener una lista de características descriptivas, por ejemplo [“Blanco”, “Buen Estado”, “Cómodo”, “En taller”]. A cada vehículo se le puede agregar una característica, o preguntar si posee la misma. En el caso de la flota, si le preguntan si posee una característica responde afirmativamente si al menos uno de sus elementos la posee.

Continuamos

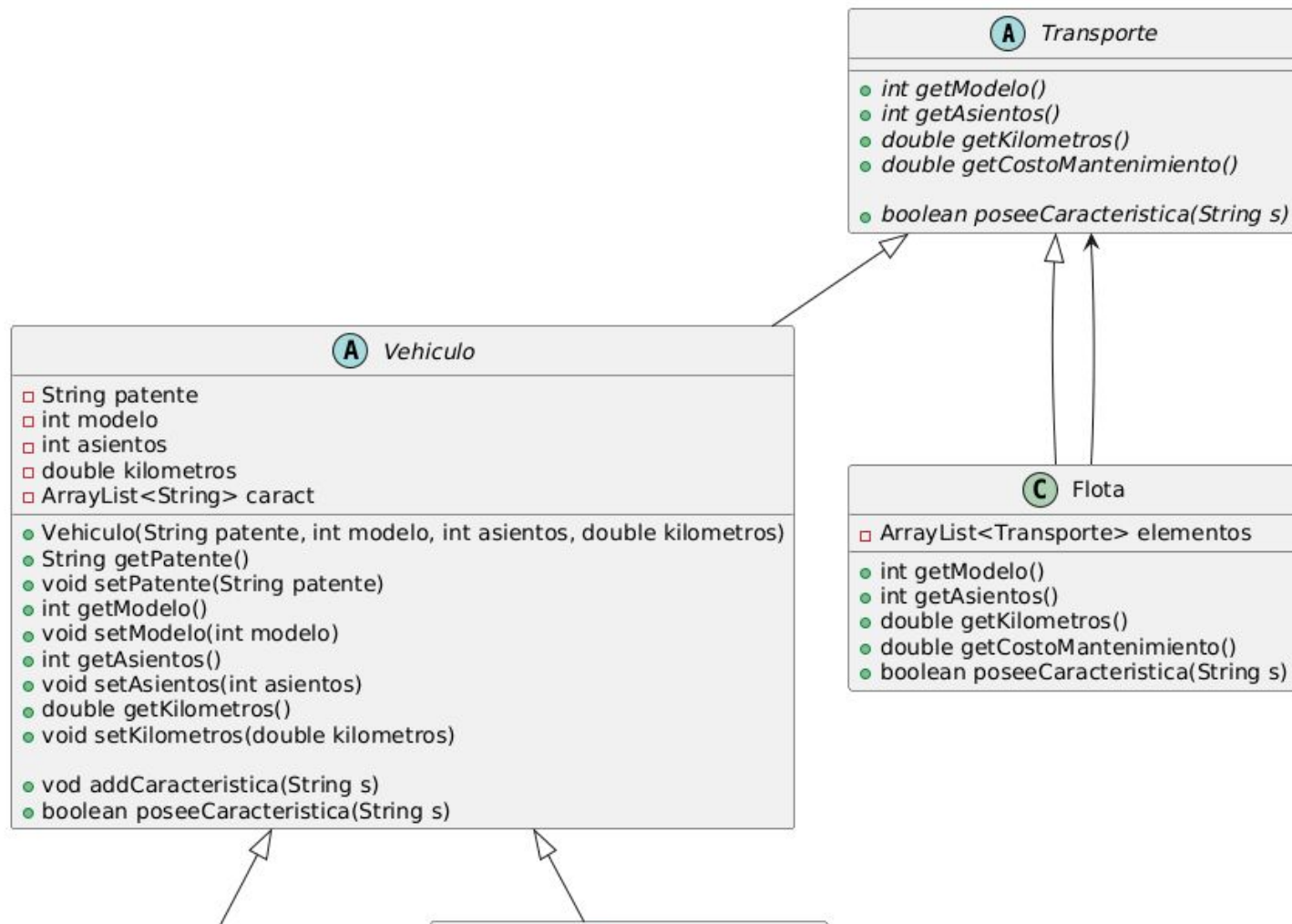
Para facilitar la descripción, cada **vehículo** debe contener una **lista de características descriptivas**, por ejemplo ["Blanco", "Buen Estado", "Cómodo", "En taller"]. A cada vehículo **se le puede agregar una característica**, o **preguntar si posee la misma**. En el caso de la **flota**, si le **preguntan si posee una característica responde** afirmativamente si al menos uno de sus elementos la posee.

Detalle

- + Vehiculo tiene una lista de características (String)
- + Tiene un método addCaracteristica(String)
- + Tiene un método poseeCaracteristica(String)
- + FLota tiene un método poseeCaracteristica(String)
- + Debo abstraer poseeCartacteristica(String) en Transporte

Agregados

— — —



Cambios

```
//Transporte  
public abstract boolean poseeCaracteristica(String s);
```

```
//Vehiculo  
private ArrayList<String> caract;  
public void addCaracteristica(String s){  
    if(!caract.contains(s))  
        caract.add(s);  
}  
public boolean poseeCaracteristica(String s){  
    return caract.contains(s);  
}
```

```
//Flota  
public boolean poseeCaracteristica(String s){  
    for(Transporte ee: elementos){  
        if (ee.poseeCaracteristica(s)  
            return true;  
        }  
    return false;  
}
```



Búsquedas

Lex, el alcalde de metrópoli, desea contar con un sistema que le permita obtener una lista de todos los vehículos (solo vehículos) que cumplan con distintas condiciones:

- modelo mayor a 2021.
- que tengan más de 100.000 kilómetros.
- que tengan la característica “Buen estado” y modelo mayor a 2010.

Los anteriores son algunos ejemplos de posibles condiciones se pueden agregar nuevas e incluso combinaciones lógicas de las mismas. Las flotas NO se incluyen en el listado pero sí los vehículos que posean y cumplan lo solicitado (sin importar el nivel de anidamiento).

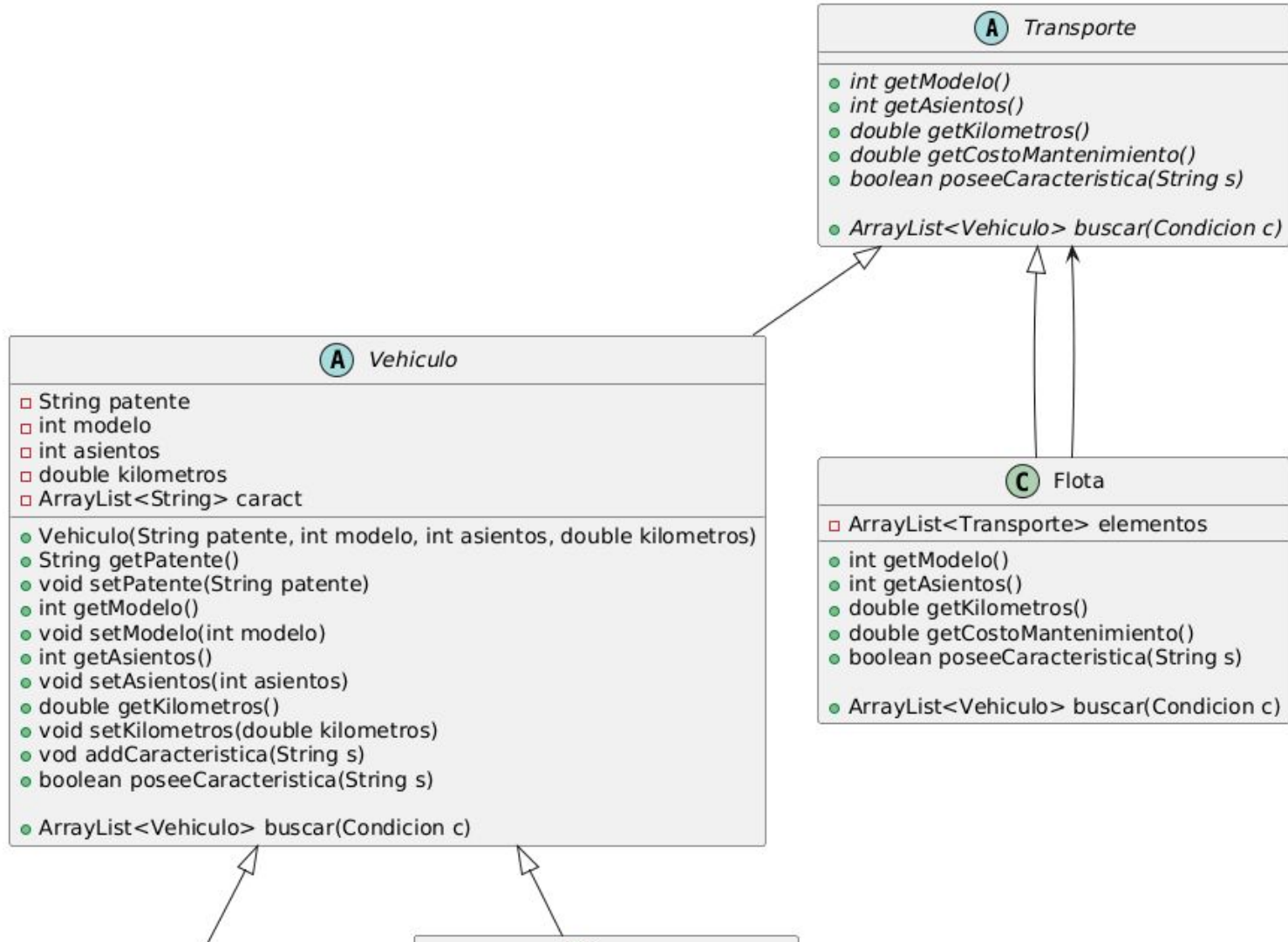
Búsquedas

— — —

- + Se debe devolver un listado de Vehiculos
- + Aparecen las “Condiciones” de búsqueda que permiten abstraer la pregunta si un vehiculo se agrega o no en la busqueda
- + Se debe abstraer la búsqueda a cualquier transporte

Búsquedas

— — —



Búsquedas

```
//Transporte  
public abstract ArrayList<Vehiculo> buscar(Condicion c)
```

```
//Vehiculo  
public ArrayList<Vehiculo> buscar(Condicion cc){  
    ArrayList<Vehiculo> salida = new ArrayList<Vehiculo>();  
    if (cc.cumple(this)) {  
        salida.add(this);  
    }  
    return salida;  
}
```



Búsquedas

```
//Transporte
```

```
public ArrayList<Vehiculo> buscar(Condicion c)
```

```
File  
public ArrayList<Vehiculo> buscar(Condicion cc){  
    ArrayList<Vehiculo> salida = new ArrayList<Vehiculo>();  
    for(Transporte ee: elems) {  
        if (cc.cumple(ee))  
            salida.add(aux);  
    }  
    return salida;  
}
```

No propaga la búsqueda, es sólo
de un nivel



Búsquedas

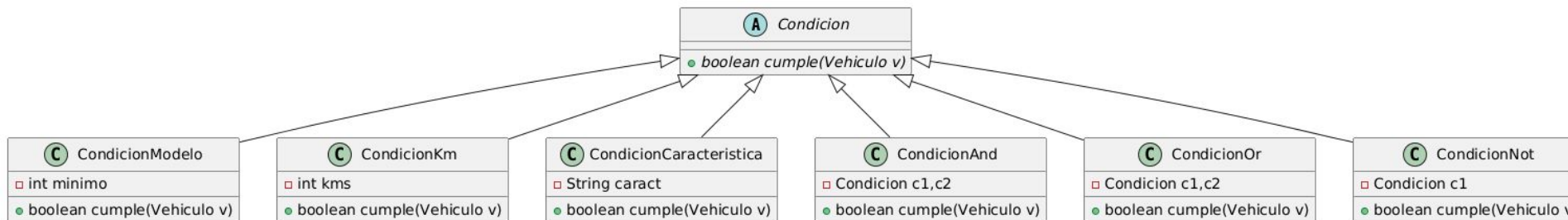
```
//Transporte  
public abstract ArrayList<Vehiculo> buscar(Condicion c)
```

```
//Flota  
public ArrayList<Vehiculo> buscar(Condicion cc){  
    ArrayList<Vehiculo> salida = new ArrayList<Vehiculo>();  
    for(Transporte ee: elems) {  
        ArrayList<Vehiculo> aux = ee.buscar(cc);  
        salida.addAll(aux);  
    }  
    return salida;  
}
```

Propago la búsqueda y voy
uniendo los resultados en
Salida



Condiciones



```
public abstract class Condicion {  
    public abstract boolean cumple(Vehiculo vv);  
}
```

Condiciones

```
public class CondicionKm extends Condicion {  
    private int kms;  
    public boolean cumple(Vehiculo vv) {  
        return vv.getKilometros() > kms;  
    }  
}
```

```
public class CondicionCaracteristica extends Condicion {  
    private String buscada;  
    public boolean cumple(Vehiculo vv) {  
        return vv.tieneCaracteristica(buscada);  
    }  
}
```

```
public class CondicionModelo extends Condicion {  
    private int minimo;  
    public boolean cumple(Vehiculo vv) {  
        return vv.getModelo() > minimo;  
    }  
}
```



Condiciones

```
public class CondicionAnd extends Condicion {
    private Condicion c1,c2;
    public boolean cumple(Vehiculo vv) {
        return c1.cumple(vv) && c2.cumple(vv);
    }
}

public class CondicionOr extends Condicion {
    private Condicion c1,c2;
    public boolean cumple(Vehiculo vv) {
        return c1.cumple(vv) || c2.cumple(vv);
    }
}

public class CondicionNot extends Condicion {
    private Condicion c1;
    public boolean cumple(Vehiculo vv) {
        return !c1.cumple(vv);
    }
}
```



Ultima Parte

En virtud de la adopción del sistema, se debe proveer mecanismos que permitan incorporar las siguientes flotas/vehiculos

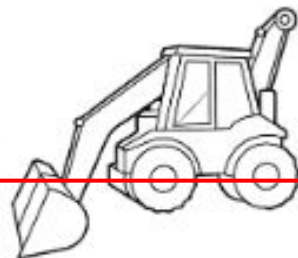
- Flota limitada de modelo: solo permite incorporar a la misma elementos que tengan un modelo mayor a 2024.
- Combis escolares: tienen una patente, un modelo, kilometraje y normalmente tienen 20 asientos (aunque hay algunas con capacidades mayores). El costo diario de mantenimiento de las mismas es de \$6.000 por asiento.
- Flota de poco uso: solo permite incorporar a la misma elementos que no tengan más de 10.000 kilómetros.
- Remises: son similares a los taxis pero se diferencian en que el extra de costo de mantenimiento, se suma si el año del vehículo es anterior a 2020.
- Maquinaria Vial: posee patente, modelo, kilometros, cantidad de asientos y horas de trabajo realizadas. El costo de mantenimiento diario es \$3200 por las horas trabajadas, aunque este valor (\$3200) puede cambiar a futuro y **es el mismo para todas las maquinarias viales**.
- Flota aesthetic: solo permite incorporar a la misma elementos que posean la característica "Beige".



Ultima Parte

En virtud de la adopción del sistema, se debe proveer mecanismos que permitan incorporar las siguientes flotas/vehiculos

- Flota limitada de modelo: solo permite incorporar a la misma elementos que tengan un modelo mayor a 2024.
- Combis escolares: tienen una patente, un modelo, kilometraje y normalmente tienen 20 asientos (aunque hay algunas con capacidades mayores). El costo diario de mantenimiento de las mismas es de \$6.000 por asiento.
- Flota de poco uso: solo permite incorporar a la misma elementos que no tengan más de 10.000 kilómetros.
- Remises: son similares a los taxis pero se diferencian en que el extra de costo de mantenimiento, se suma si el año del vehículo es anterior a 2020.
- Maquinaria Vial: posee patente, modelo, kilometros, cantidad de asientos y horas de trabajo realizadas. El costo de mantenimiento diario es \$3200 por las horas trabajadas, aunque este valor (\$3200) puede cambiar a futuro y **es el mismo para todas las maquinarias viales**.
- Flota aesthetic: solo permite incorporar a la misma elementos que posean la característica "Beige".



Ultima Parte

En virtud de la adopción del sistema, se debe proveer mecanismos que permitan incorporar las siguientes flotas/vehiculos

Flota limitada de modelo: solo permite incorporar a la misma elementos que tengan un modelo

Son tres clases similares controlan algo y si se cumple lo agregan

en una patente, un modelo, algunas con capacidades mayores por asiento.

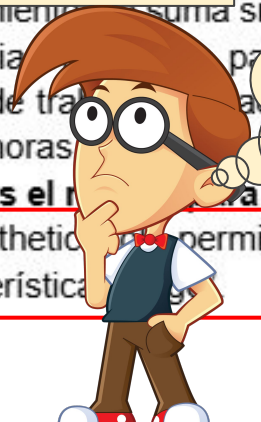
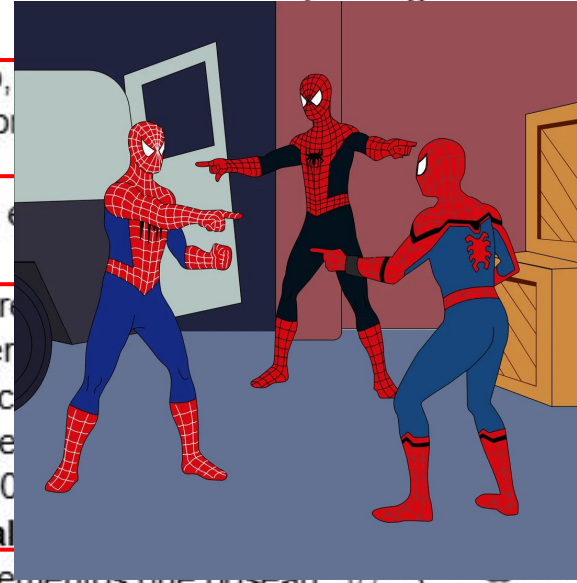
permite incorporar a la misma

a los tres se difer

- Maquinaria y horas de trabajo por las horas futuro y es el

Puedo reusar las condiciones

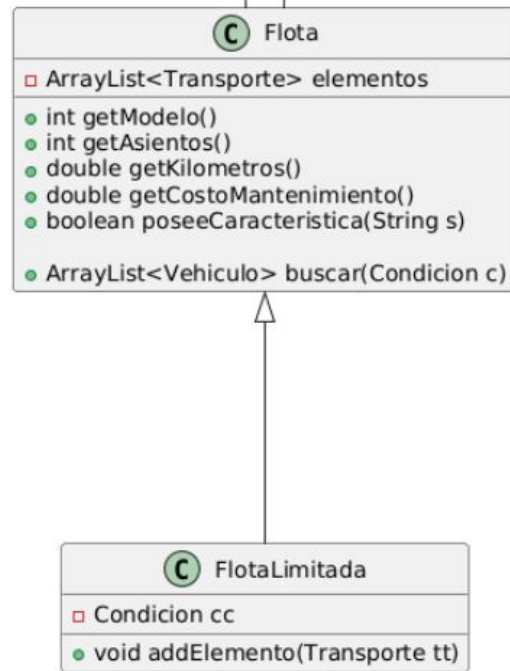
- Flota aesthetica permite incorporar a la misma elementos que posean la característica



FlotaLimitada

```
public class FlotaLimitada extends Flota {  
  
    private Condicion cc;  
    public void addElemento(Transporte tt) {  
        if (cc.cumple(tt)) {  
            super.addElemento(tt);  
        }  
    }  
}
```

Uso super en lugar de
acceder al ArrayList
de Flota



FlotaLimitada

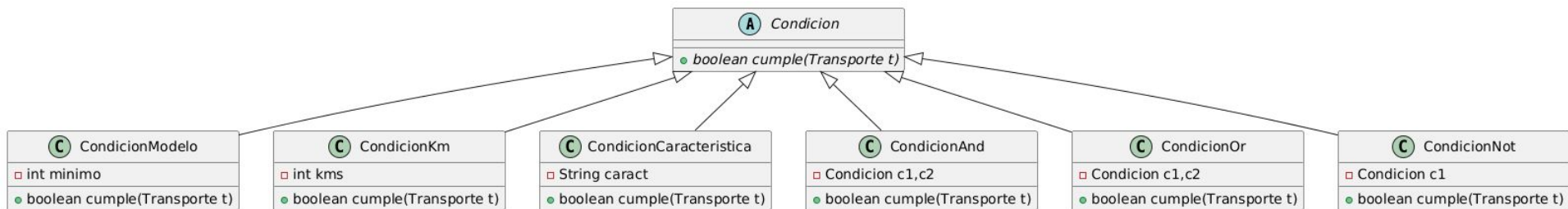
```
public class FlotaLimitada extends Flota {  
  
    private Condicion cc;  
    public void addElemento(Transporte tt) {  
        if (cc.cumple(tt)) {  
            super.addElemento(tt);  
        }  
    }  
}
```



Pero Condicion era
sobre Vehiculo

Condiciones 2.0

— — —



Como la Clase Transporte tiene los métodos getModelo, getKilometros y poseeCaracteristica no necesito mas que cambiar Vehiculo por Transporte (en todas las clases Condicion)

Ultima Parte

En virtud de la adopción del sistema, se debe proveer mecanismos que permitan incorporar las siguientes flotas/vehiculos

- Flota limitada de modelo: solo permite incorporar a la misma elementos que tengan un modelo mayor a 2024.
- Combis escolares: tienen una patente, un modelo, kilometraje y normalmente tienen 20 asientos (aunque hay algunas con capacidades mayores). El costo diario de mantenimiento de las mismas es de \$6.000 por asiento.
- Flota de poco uso: solo permite incorporar a la misma elementos que no tengan más de 10.000 kilómetros.
- Remises: son similares a los taxis pero se diferencian en que el extra de costo de mantenimiento es menor a \$2000.
- Maquinaria pesada: solo permite incorporar a la misma elementos que tengan una capacidad de asientos menor a 20. El costo diario de mantenimiento es \$3200 y puede cambiar a diario.
- Flota aesthetic: solo permite incorporar a la misma elementos que posean la característica "Beige".

No es una clase nueva, ya que es un
EJEMPLO INSTANCIA de Colectivo



Ultima Parte

En virtud de la adopción del sistema, se debe proveer mecanismos que permitan incorporar las siguientes flotas/vehiculos

- Flota limitada de modelo: solo permite incorporar a la misma elementos que tengan un modelo mayor a 2024.
- Combis escolares: tienen una patente, un modelo, kilometraje y normalmente tienen 20 asientos (aunque hay algunas con capacidades mayores). El costo diario de mantenimiento de las mismas es de \$6.000 por asiento.
- Flota de poco uso: solo permite incorporar a la misma elementos que no tengan más de 10.000 kilómetros.
- Remises: son similares a los taxis pero se diferencian en que el extra de costo de mantenimiento, se suma si el año del vehículo es anterior a 2020.
- Maquinaria Vial: posee patente, modelo, kilometraje, cantidad de asientos y horas de trabajo realizadas. El costo diario es de \$10.000 por hora de trabajo por las horas trabajadas. El costo de mantenimiento es de \$10.000 por hora de trabajo futuro y **es el mismo para todos los vehículos**.
- Flota aesthetic: solo permite incorporar a la misma elementos que posean la característica "Beige".

Es un ejemplo del taxi, no es clase nueva

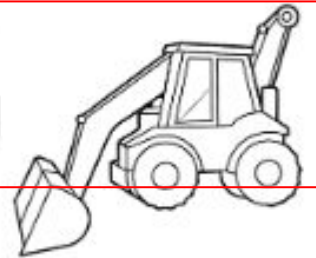


Ultima Pa

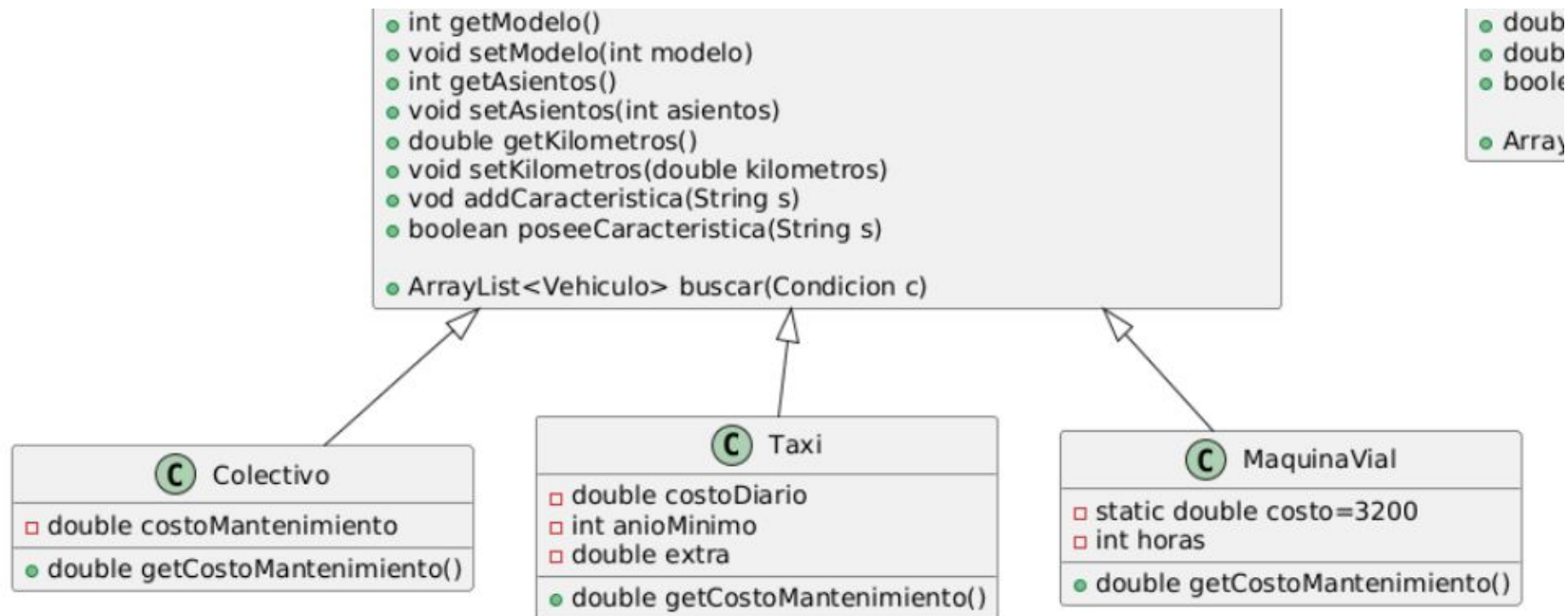
En virtud de
flotas/vehículo

- delo: solo permite i
- tienen una patente y algunas con capa 000 por asiento.
- solo permite incorporar a la misma elementos que no tengan más de 10.000
- ares a los taxis pero se diferencian en que el extra de costo de mantenimiento, se suma si el año del vehículo es anterior a 2020.
- Maquinaria Vial: posee patente, modelo, kilometros, cantidad de asientos y horas de trabajo realizadas. El costo de mantenimiento diario es \$3200 por las horas trabajadas, aunque este valor (\$3200) puede cambiar a futuro y **es el mismo para todas las maquinarias viales**.
- Flota aesthetic: solo permite incorporar a la misma elementos que posean la característica "Beige".

Agrega algo distinto, es una
clase nueva

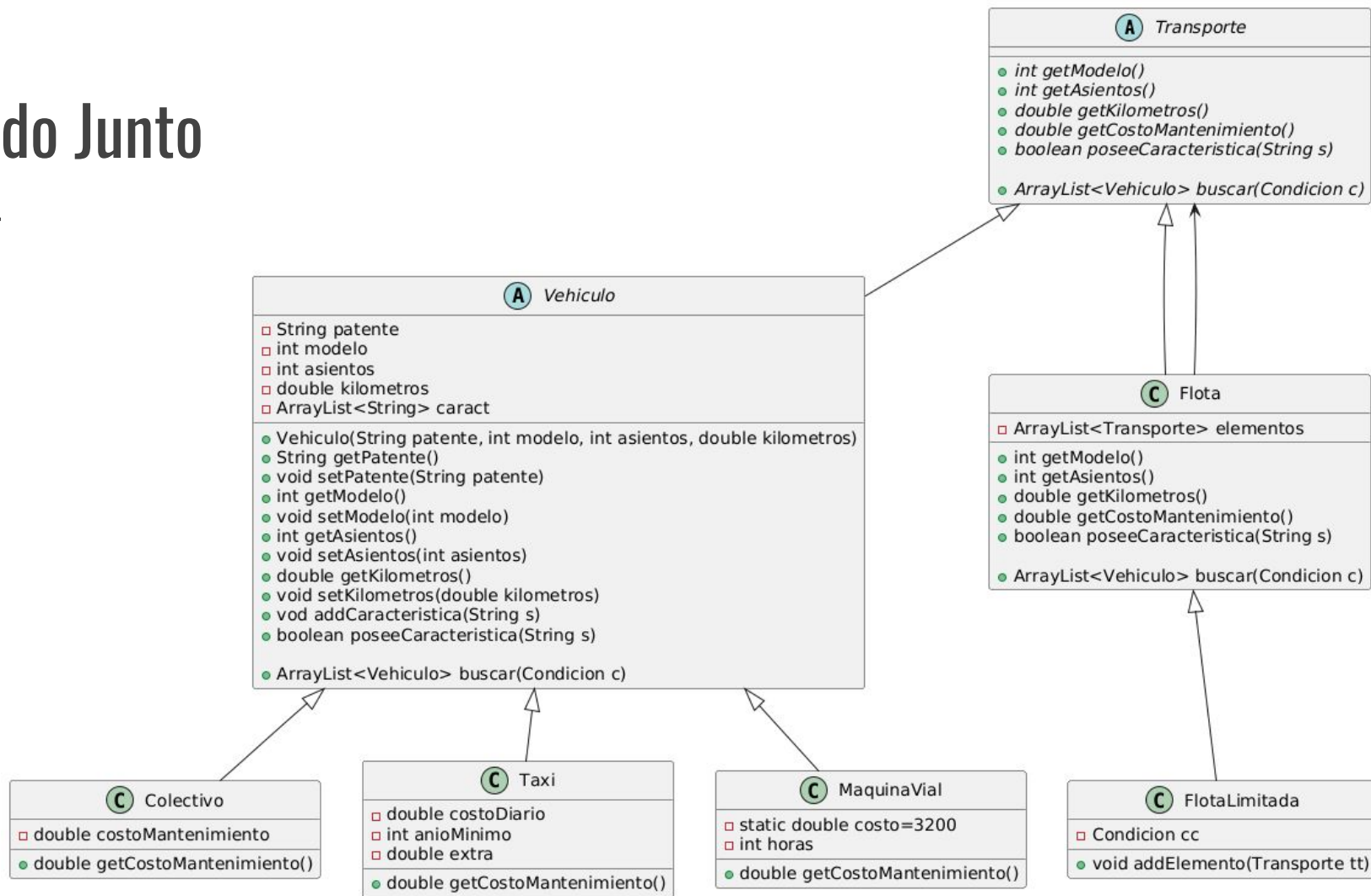


MaquinaVial



Todo Junto

--



Condiciones

— — —

